

Forex Advisory

Technical Documentation

Architecture & API Reference

A Streamlit application for analyzing historical foreign-exchange rates

Project: ZPI2025_IO1 - Segmentation fault
Version 2.x · June 2026

Table of Contents

Table of Contents	2
1. Introduction	3
1.1 Purpose & Scope	3
1.2 Technology Stack	3
2. Architecture & Design	4
2.1 Architectural Overview	4
2.2 Project Structure	4
2.3 Component Diagram	4
2.4 Data Flow	5
2.5 User Interaction Flow	6
2.6 Key Design Decisions	7
3. API Reference	8
3.1 Entry Points	8
3.2 Utility Layer - src/utls	8
3.3 Presentation Layer - src/ui	9
4. Build, Test & Deployment	11
4.1 Prerequisites & Running	11
4.2 Testing	11
4.3 Continuous Integration & Delivery	11
4.4 Data Source & Limitations	11

1. Introduction

Forex Advisory is a Streamlit web application for analyzing historical foreign-exchange rates. It fetches official average rates from the National Bank of Poland (NBP) Web API and presents statistical insights, price trends, and cross-rate change distributions for more than thirty world currencies, each quoted against the Polish Złoty (PLN).

This document describes the system from two perspectives. The architecture section explains how the application is structured, how data flows through it, and how the pieces fit together. The API reference section documents every module and public function in the source tree, its purpose, parameters, return values, and error behaviour.

1.1 Purpose & Scope

The application targets users who want a lightweight, browser-based tool for exploring exchange-rate history without spreadsheets or external data wrangling. It offers two analysis modes:

- **Base Price Analysis:** examines a single currency against PLN: a price chart, the latest price with session-over-session change, statistical indicators, and a count of rising / steady / falling sessions.
- **Forex Pair Analysis:** compares two currencies by computing their cross-rate (currency 1 ÷ currency 2) and plotting the distribution of day-to-day changes as a histogram.

1.2 Technology Stack

Layer	Technology	Role
Web UI	Streamlit 1.57	Page layout, navigation, widgets, rendering
Charts	Altair 6.1	Declarative line, bar, and histogram charts
Data processing	pandas 3.0 / NumPy 2.4	DataFrames, statistics, histogram binning
HTTP	requests 2.33	Calls to the NBP Web API
Data source	NBP Web API (Table A)	Official PLN average exchange rates
Testing	pytest 9.0	Unit, integration, and acceptance tests
CI / CD	GitHub Actions	Linting, multi-version tests, release builds

2. Architecture & Design

2.1 Architectural Overview

The application follows a simple, layered architecture with a clear separation between presentation and logic. There is no persistent storage: every interaction triggers a fresh request to the NBP API, and all computation happens in memory on the server side of the Streamlit session.

Three layers can be distinguished:

1. **Entry / navigation layer:** `app.py` and `init.py` bootstrap Streamlit, configure the page, render the logo, and route the user between the two analysis pages.
2. **Presentation (UI) layer:** the modules under `src/ui/` define each page: they collect user input, orchestrate calls into the utility layer, and render charts, tables, and cards.
3. **Logic (utility) layer:** the modules under `src/utls/` are pure, framework-agnostic functions for fetching data and computing statistics. They have no Streamlit dependency and are independently unit-testable.

This separation is deliberate: the utility layer contains all the testable business logic and knows nothing about the UI, while the UI layer contains no statistics of its own. The CI test suite exercises the utility layer directly.

2.2 Project Structure

```

.
├── app.py                # Entry point - launches Streamlit on init.py
├── init.py               # Page config, logo, navigation between pages
├── requirements.txt       # Pinned Python dependencies
├── assets/
│   └── ForexAdvisory.svg # App logo
├── diagrams/              # UML diagrams (activity, components, sequence)
├── src/
│   ├── ui/
│   │   ├── base_price_analysis.py # Base Price Analysis page
│   │   └── forex_pair_analysis.py # Forex Pair Analysis page
│   └── utls/
│       ├── exchange_rates.py      # Fetches rates from the NBP API
│       ├── statistical_measures.py # Median, mode, std dev, CV
│       ├── session_analysis.py     # Rising / steady / falling counts
│       └── change_distribution_analysis.py # Cross-rate change distribution
└── tests/                    # pytest unit / integration / acceptance tests

```

2.3 Component Diagram

The component diagram shows the static structure of the system - how the entry point, the two UI pages, the utility modules, and the external NBP API relate to one another.

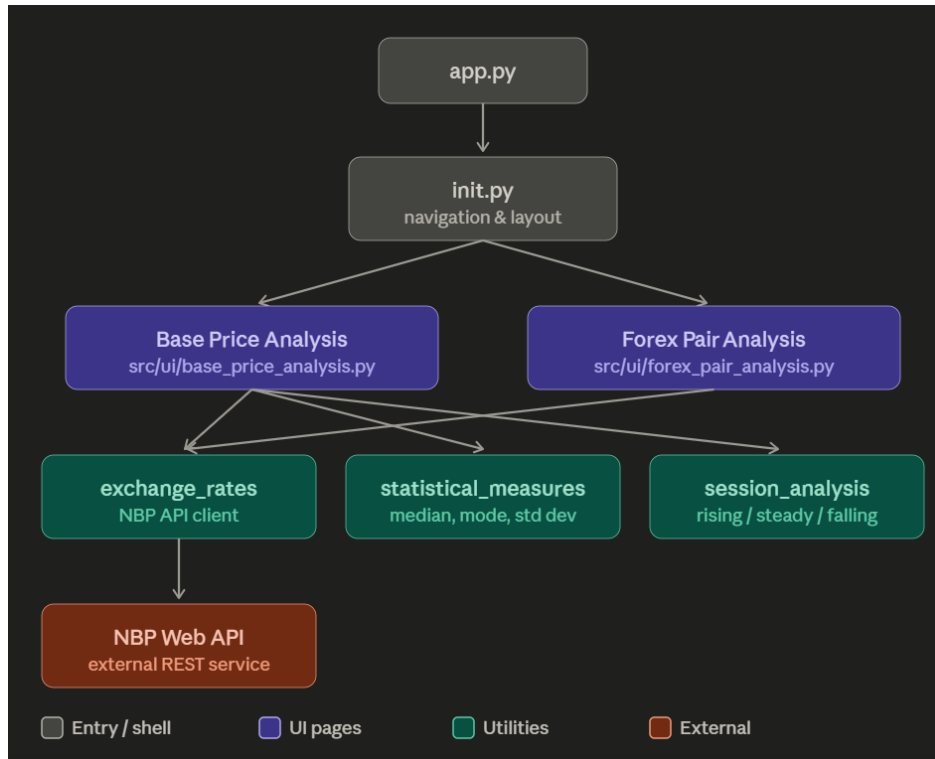


Figure 1 - Component diagram

2.4 Data Flow

A typical request proceeds as follows. The user selects an analysis period, a start date, and a currency (or pair) in the page controls. The UI module computes the end date from the period, formats the dates as YYYY-MM-DD strings, and calls `get_exchange_rates` for each currency. That function validates the date range, queries the NBP API, and returns a tidy two-column DataFrame of date and rate.

The UI module then passes the DataFrame to the relevant utility functions - `statistical_measures` and `session_analysis` on the Base Price page, or the cross-rate / histogram logic on the Forex Pair page - and renders the results with Altair charts and Streamlit dataframes. Errors raised anywhere in the chain are caught in the page's try/except block and shown to the user with `streamlit.error`.

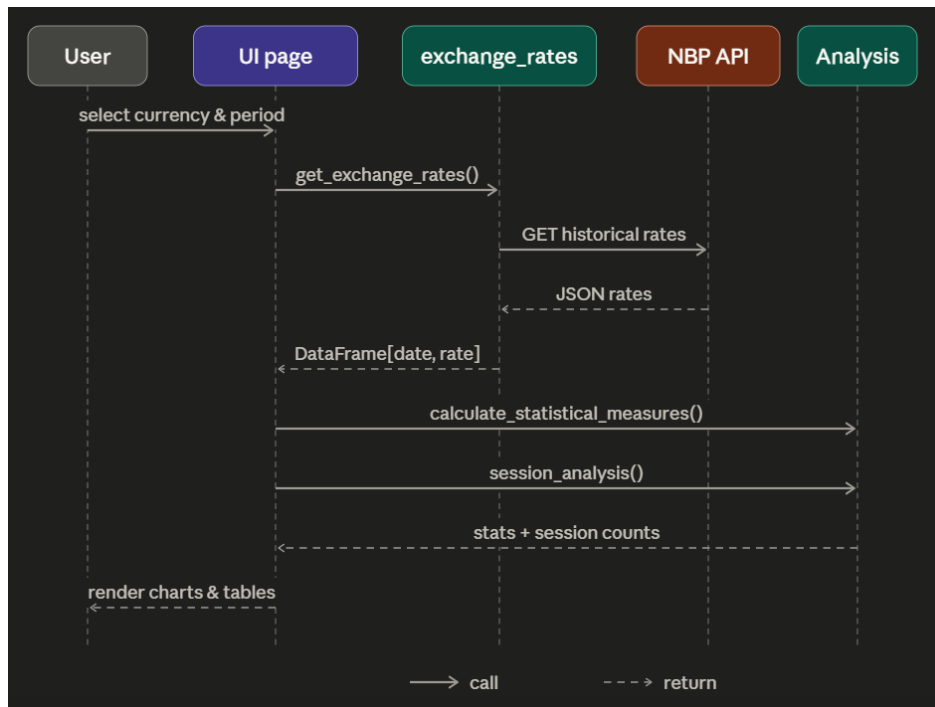


Figure 2 - Sequence diagram: a Base Price Analysis request

2.5 User Interaction Flow

The activity diagram below captures the control flow of a single page render, including the validation branches and the empty-data and error short-circuits that return early without rendering charts.

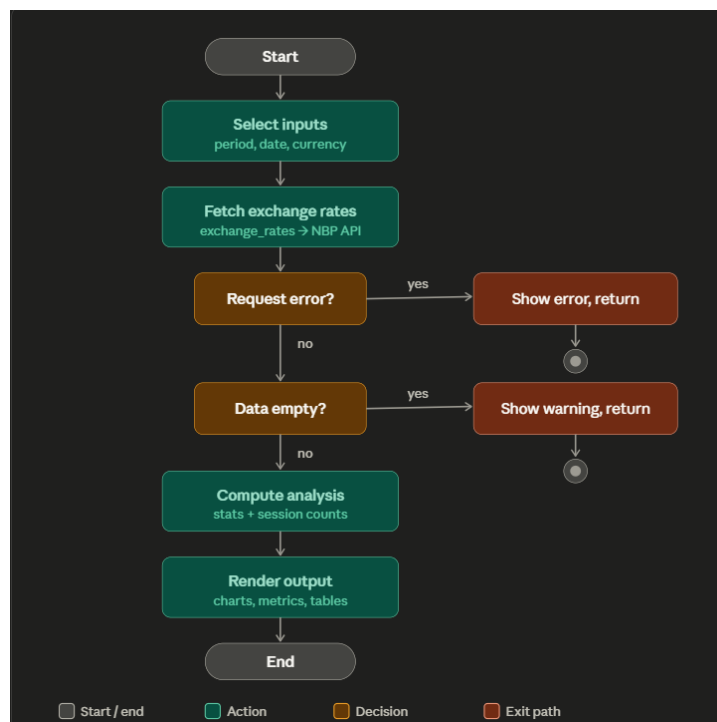


Figure 3 - Activity diagram

2.6 Key Design Decisions

Decision	Rationale
Stateless, no database	Rates are always fetched live from NBP; this keeps the app simple and the data authoritative, at the cost of a network round-trip per interaction.
Pure utility layer	Business logic is decoupled from Streamlit so it can be unit-tested in isolation and reused across both pages.
Currency list per page	Each UI page defines its own CURRENCIES and PERIODS constants, keeping pages self-contained (at the cost of some duplication).
Altair for charts	Declarative grammar-of-graphics integrates cleanly with Streamlit and pandas DataFrames.
PyInstaller distribution	The CD pipeline ships a standalone executable so non-technical users can run the app without a Python setup.

3. API Reference

This section documents every module in the source tree. For each public function it lists the signature, parameters, return value, and the exceptions it may raise.

3.1 Entry Points

app.py

The distributable entry point. It rewrites `sys.argv` to invoke the Streamlit CLI on `init.py` and delegates to `streamlit.web.cli.main()`. This indirection lets the app be packaged as a single executable by PyInstaller while still launching through Streamlit.

```
python app.py          # via the entry point
streamlit run init.py  # equivalently, directly
```

init.py

Configures the page and provides navigation between the two analysis pages. It sets the page title, icon, and wide layout; registers the two pages with `streamlit.Page`; renders the SVG logo; and draws two navigation buttons that highlight the active page and switch between them.

`render_svg(path: str, width: int) -> None`

Reads an SVG file, base64-encodes it, and embeds it as an inline `<img>` tag via `streamlit.markdown` so the logo renders at a fixed pixel width.

3.2 Utility Layer - src/utlis

Pure, framework-agnostic functions. None of these import Streamlit; all operate on pandas data structures and are covered by the test suite.

exchange_rates.py

`get_exchange_rates(currency: str, start_date: str, end_date: str) -> DataFrame`

Fetches historical average exchange rates for a single currency over a date range from the NBP API (Table A). Returns a `DataFrame` with two columns, date (datetime) and rate (float).

Parameters

Parameter	Type	Description
currency	str	Currency code in ISO 4217 format (e.g. USD). Lower-cased before the request.
start_date	str	Start date as 'YYYY-MM-DD'.
end_date	str	End date as 'YYYY-MM-DD'.

Behaviour & validation

- Parses both dates; the earliest valid start date is 2 January 2002 (`FIRST_VALID_DATE`).
- Builds the endpoint `.../exchangerates/rates/a/{currency}/{start}/{end}/?format=json` and issues a GET request.

- Selects the effectiveDate and mid fields from the response, renames them to date and rate, and converts date to datetime.

Raises

Exception	Condition
ValueError	start_date is before 2 January 2002.
ValueError	end_date is in the future.
ValueError	end_date is before start_date.
ValueError	HTTP status is not 200, or the API returns no rates.

statistical_measures.py

calculate_statistical_measures(dataset) -> dict[str, float | None]

Computes summary statistics over a series of values. Accepts a DataFrame (must contain a rate column), a Series, or a list of floats. Returns a dictionary with the keys:

Key	Meaning
median	Median of the values.
mode	Most frequent value (first mode), or None if undefined.
standard_deviation	Sample standard deviation.
coefficient_of_variation	Std dev ÷ mean, or None when the mean is zero.

Raises ValueError if a DataFrame is passed without a rate column, or if the dataset is empty.

session_analysis.py

session_analysis(exchange_rates: DataFrame) -> dict[str, int]

Counts how many trading sessions rose, fell, or stayed flat by taking the first difference of the rate column. Returns a dictionary keyed by 'Rising session', 'Falling session', and 'Steady session'.

- Raises ValueError if the input is None or empty.
- Returns all-zero counts when fewer than two rows are present (no session-over-session change can be computed).

change_distribution_analysis.py

change_distribution_analysis(data1: DataFrame, data2: DataFrame) -> list

Computes the distribution of day-to-day rate changes for two currencies. For each input it takes the first difference of the rate column, rounds to two decimals, and counts occurrences. Returns a list of two dictionaries mapping rounded change → frequency.

- Raises ValueError if either argument is None or empty.

3.3 Presentation Layer - src/ui

Each UI module exposes a single `render()` function that Streamlit calls to draw the page. Both modules define module-level `PERIODS` and `CURRENCIES` constants describing the selectable analysis windows and the 30 supported currencies.

base_price_analysis.py

`render()` -> None

Draws the Base Price Analysis page. It collects the analysis period, start date, and currency; derives the end date; and fetches rates via `get_exchange_rates` inside a spinner, surfacing any error with `streamlit.error`. On success it renders:

- A price line chart (Altair) with a dashed reference rule at the current price, beside a Current Price card showing the latest rate and its absolute and percentage change, colour-coded green for gains and red for losses.
- A Statistical Indicators table built from `calculate_statistical_measures` (median, mode, standard deviation, coefficient of variation).
- A Price Changes panel built from `session_analysis`, shown as both a table (Upwards / No Change / Downwards) and a horizontal bar chart.

Supported periods: 1 Week, 2 Weeks, 1 Month, 1 Quarter, Half a Year, 1 Year.

forex_pair_analysis.py

`render()` -> None

Draws the Forex Pair Analysis page. It collects the period, start date, and two currencies (defaulting the second to the next in the list), then fetches both series via `get_exchange_rates`. It merges the two DataFrames on date, computes the cross-rate ($\text{rate}_1 \div \text{rate}_2$), and takes the first difference of that cross-rate.

- Bins the changes with `numpy.histogram(..., bins="auto")` and renders a horizontal histogram (Altair) of frequency by change-range bin, alongside a companion table of bin ranges and counts.
- Short-circuits with a warning when either series is empty, when there are no overlapping dates, or when there are too few points to compute changes.

Supported periods: Monthly, Quarterly.

4. Build, Test & Deployment

4.1 Prerequisites & Running

The application requires Python 3.11, 3.12, or 3.13 and an internet connection (it calls the live NBP API). Dependencies are pinned in requirements.txt.

```
pip install -r requirements.txt
python app.py          # or: streamlit run init.py
# App opens at http://localhost:8501
```

4.2 Testing

Unit tests live under tests/ and target the utility layer (exchange rates, statistical measures, session analysis, change distribution); integration and acceptance test packages are also present. Run the suite with pytest:

```
pytest tests/
```

4.3 Continuous Integration & Delivery

Two GitHub Actions workflows automate quality checks and releases:

Workflow	Trigger	Actions
CI (ci.yml)	PRs to main, develop, release	Runs flake8 linting and the pytest suite across Python 3.11, 3.12, and 3.13.
CD (cd.yml)	Push to release	Builds a standalone executable with PyInstaller, auto-increments the v2.0.x tag, and publishes a GitHub Release with the artifact.

4.4 Data Source & Limitations

- Rates come from the NBP Table A average rates, quoted against PLN.
- The earliest available date is 2 January 2002; the end date cannot be in the future.
- The NBP API publishes rates only on banking days, so non-trading days are absent from the data, a deliberate gap, not missing data.

Source: <https://api.nbp.pl/>